
Proceso de aprendizaje en redes neuronales

Así como hay muchos diferentes tipos de aprendizaje y enfoques para el aprendizaje humano, también podemos decir sobre las máquinas. Para garantizar que una máquina pueda aprender de la experiencia, es importante definir las mejores metodologías disponibles en función de los requisitos específicos del trabajo. Esto a menudo significa elegir técnicas que funcionen para el presente caso y evaluarlas de vez en cuando para determinar si necesitamos probar algo nuevo.

Hemos visto los conceptos básicos de las redes neuronales en los acetatos anteriores, y también dos implementaciones simples que usan R. En esta parte, trataremos el proceso de aprendizaje, es decir, cómo entrenar, probar e implementar un modelo de aprendizaje automático de redes neuronales. La fase de entrenamiento se utiliza para aprender, para adaptarse a los parámetros de las redes neuronales. La fase de prueba se utiliza para evaluar el rendimiento de las redes neuronales totalmente entrenadas. Finalmente, en la fase de implementación, los datos reales se pasan a través del modelo para obtener la predicción.

La siguiente es la lista de conceptos cubiertos en estas notas:

- Proceso de aprendizaje
- Aprendizaje supervisado
- Aprendizaje sin supervisión
- Capacitación, prueba e implementación de un modelo.
- Evaluación de la medición de error de medición y ajuste fino; precisión de la medición de un modelo
- Modelo de aprendizaje supervisado utilizando redes neuronales
- Modelo de aprendizaje no supervisado utilizando redes neuronales
- Propagación hacia atrás

Al final de estas notas, entenderemos los conceptos básicos del proceso de aprendizaje y cómo implementarlo en el entorno R. Descubriremos diferentes tipos de algoritmos para implementar una red neuronal. Aprenderemos cómo entrenar, probar e implementar un modelo. Sabremos cómo realizar un procedimiento de valoración correcto.

¿Qué es el aprendizaje automático?

¿Qué queremos decir con el término **aprendizaje automático o máquina**? La definición es bastante difícil, para hacerlo, estamos pidiendo ayuda a un gran campo de científicos. Podemos mencionar la cita de un pionero de la inteligencia artificial:

“Campo de estudio que brinda a las computadoras la capacidad de aprender sin ser programado explícitamente”.

- Arthur Samuel

El aprendizaje automático se trata de entrenar un modelo o un algoritmo con datos y luego usar el modelo para predecir datos nuevos. Por ejemplo, a un niño pequeño se le enseña a caminar desde su fase de gatear. Inicialmente, los padres del niño sostienen la mano del niño para ayudarlo a levantarse, y se le enseña a través de los datos que se proporcionan. Sobre la base de estos procedimientos, si se presenta un obstáculo en el camino del niño o si hay un giro en alguna parte, el niño puede navegar solo después del entrenamiento. Los datos utilizados para la capacitación son los datos de capacitación y el destinatario continúa aprendiendo incluso después de la capacitación formal.

A las máquinas también se les puede enseñar como a los niños pequeños a hacer una tarea basada en el entrenamiento. Primero, alimentamos suficientes datos para decirle a la máquina qué debe hacerse en qué circunstancias. Después del entrenamiento, la máquina puede funcionar automáticamente y también puede aprender a afinarse. Este tipo de entrenamiento de la máquina se llama **aprendizaje automático**.

La principal diferencia entre el aprendizaje automático y la programación es que no hay codificación/programación involucrada en el aprendizaje automático, mientras que la programación consiste en darle a la máquina un conjunto de instrucciones para realizar. En el aprendizaje automático, los datos son la única entrada proporcionada y el modelo se basa en el algoritmo que hemos decidido utilizar.

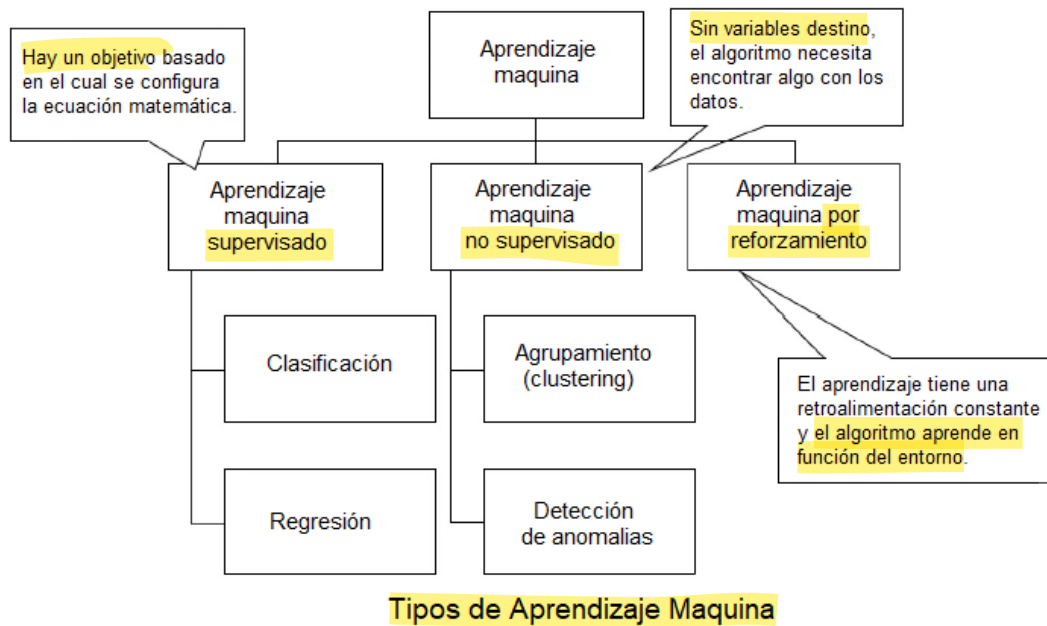
El algoritmo a utilizar se basa en varios factores de los datos: las características (o variables independientes), el tipo de variable(s) dependiente(s), la precisión del modelo y la velocidad de entrenamiento y predicción del modelo.

Según las variables independientes de los datos de aprendizaje automático, hay tres formas diferentes de entrenar un modelo:

- Aprendizaje supervisado
- Aprendizaje sin supervisión
- Aprendizaje reforzado

La siguiente figura muestra los diferentes algoritmos para entrenar un modelo de aprendizaje automático:

En las siguientes secciones, los veremos uno por uno.



Aprendizaje supervisado

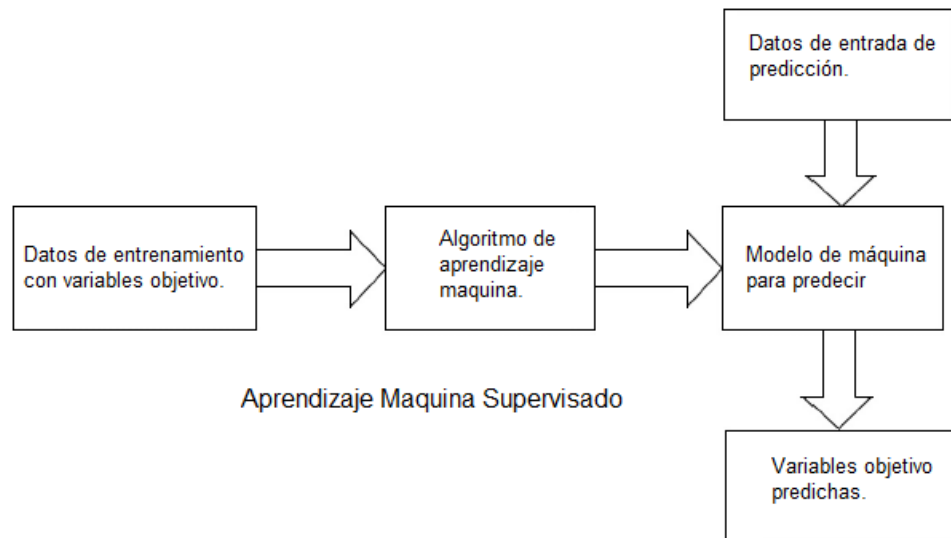
El aprendizaje supervisado es un método de aprendizaje en el que hay una parte de los datos de entrenamiento que actúa como maestro (experto) del algoritmo para determinar el modelo. A la máquina se le enseña qué aprender de los datos objetivo. Los datos objetivo, o las variables dependientes o de respuesta, son el resultado de la acción colectiva de las variables independientes.

El entrenamiento de la red se realiza con los datos objetivo y su comportamiento con patrones de datos de entrada. Las etiquetas objetivo se conocen de antemano y los datos se envían al algoritmo para derivar el modelo.

La mayor parte del uso de la red neuronal se realiza mediante aprendizaje supervisado. Los pesos y sesgos se ajustan en función de los valores de salida. El resultado puede ser categórico (como verdadero/falso o 0/1/2) o continuo (como 1, 2, 3, etc.). El modelo depende del tipo de variables de salida y, en el caso de las redes neuronales, la capa de salida se basa en el tipo de variable objetivo.

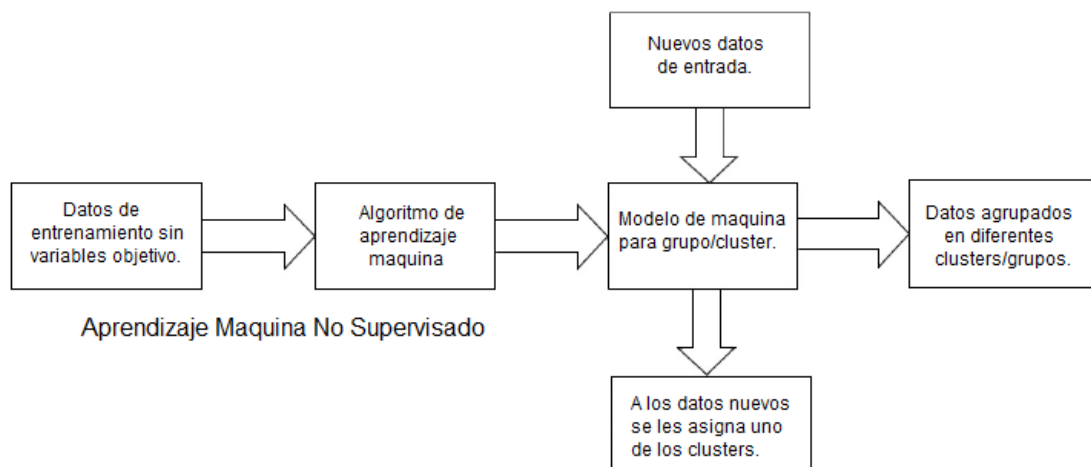
Nota: Para las redes neuronales, todas las variables independientes y dependientes deben ser numéricas, ya que una red neuronal se basa en modelos matemáticos. Depende del científico de datos convertir los datos a números para alimentarlos al modelo.

El aprendizaje supervisado se representa en el siguiente diagrama:



Aprendizaje no supervisado

En el aprendizaje no supervisado (o autoorganización), la capa de salida está entrenada para organizar los datos de entrada en otro conjunto de datos sin la necesidad de una variable objetivo. Los datos de entrada se analizan y se encuentran patrones en ellos para derivar la salida, como se muestra en la siguiente figura. Como no hay maestro (o variable objetivo), este tipo de aprendizaje se denomina aprendizaje no supervisado.



Las diferentes técnicas disponibles para el aprendizaje no supervisado son las siguientes:

- Clustering (K-means, jerárquico)
- Técnicas de asociación.
- Reducción de dimensionalidad
- Mapa de autoorganización (SOM) / redes Kohonen

Para resumir, los dos tipos principales de aprendizaje automático se muestran en la siguiente figura:

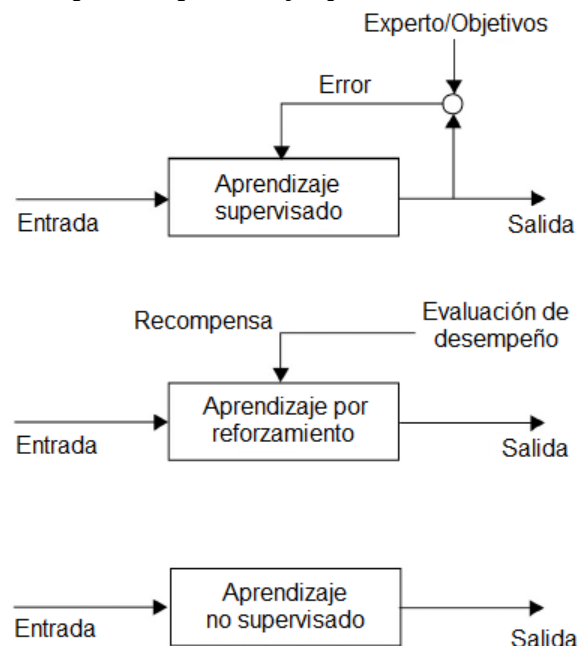


Para las redes neuronales, tenemos los dos tipos disponibles, utilizando diferentes formas disponibles en R (como se verá posteriormente).

Aprendizaje reforzado

El aprendizaje de refuerzo es un tipo de aprendizaje automático en el que **el modelo recibe retroalimentación constante para adaptarse al entorno**. Hay una evaluación de desempeño en cada paso para mejorar el modelo. Para las redes neuronales, hay un tipo especial de aprendizaje llamado Q (Q-learning), combinado con neuronas para implementar el aprendizaje de refuerzo en el mecanismo de retroalimentación de propagación hacia atrás. Los detalles están fuera del alcance de este curso y notas.

Los siguientes son los tres tipos de aprendizaje que hemos cubierto hasta ahora:



Entrenando y probando el modelo

Entrenar y probar el modelo forma la base para un mayor uso del modelo para la predicción en análisis predictivo. Dado un conjunto de datos de 100 filas de datos, que incluye las variables de predicción y respuesta, dividimos el conjunto de datos en una proporción conveniente (digamos 70:30) y asignamos 70 filas para entrenamiento y 30 filas para pruebas. Las filas se seleccionan al azar para reducir el sesgo.

Una vez que los datos de entrenamiento están disponibles, los datos se envían a la red neuronal para obtener la función universal masiva. Los datos de entrenamiento determinan los pesos, sesgos y funciones de activación que se utilizarán para llegar a la salida de la entrada.

Hasta hace poco, no podíamos decir que un peso tiene una influencia positiva o negativa en la variable objetivo. Pero ahora hemos podido arrojar algo de luz dentro de la caja negra. Por ejemplo, al graficar una red neuronal entrenada, podemos descubrir pesos sinápticos entrenados e información básica sobre el proceso de entrenamiento.

Una vez que se logra una convergencia suficiente, el modelo se almacena en la memoria y el siguiente paso es probar el modelo. Pasamos las 30 filas de datos para verificar si la salida real coincide con la salida predicha del modelo. La evaluación se utiliza para obtener varias métricas que pueden validar el modelo.

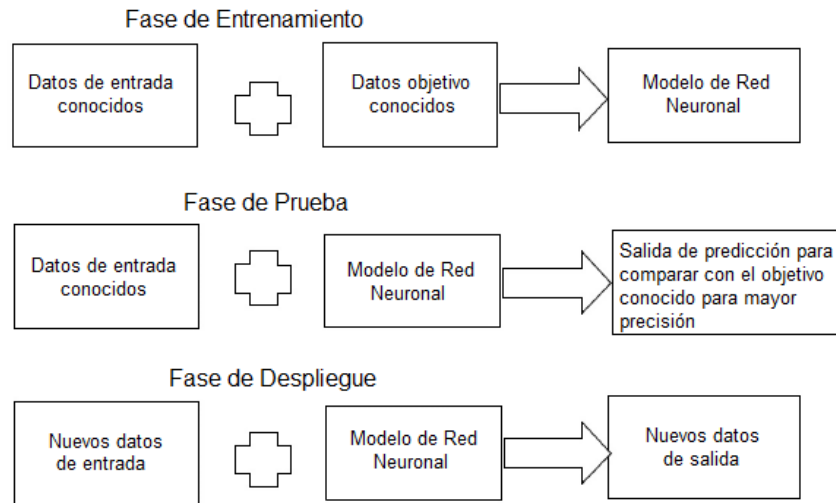
Si la precisión es demasiado cautelosa, el modelo debe reconstruirse con cambios en los datos de entrenamiento y otros parámetros pasados a la función de la red neuronal. Cubriremos más sobre las métricas de evaluación más adelante en estas notas.

Después del entrenamiento y las pruebas, se dice que el modelo se despliega, donde los datos reales se pasan a través del modelo para obtener la predicción.

Por ejemplo, el caso de uso puede ser determinar una transacción de fraude o una verificación de elegibilidad para préstamos hipotecarios basada en varios parámetros de entrada.

El entrenamiento, las pruebas y el despliegue se representan en la siguiente figura:

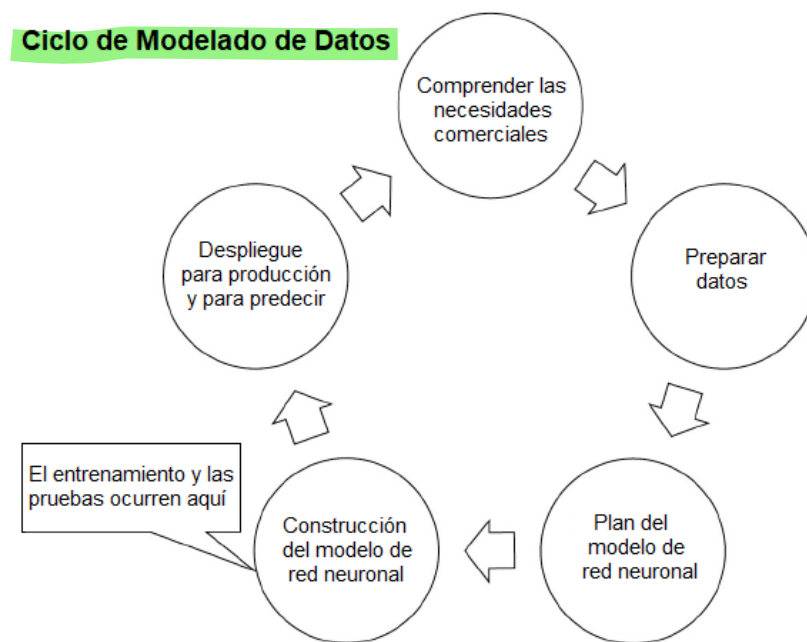
Hasta ahora, nos hemos centrado en los diversos algoritmos disponibles; Ahora es el momento de dedicarnos a los datos que representan el elemento esencial de cada análisis.



El ciclo de datos

Los datos forman un componente clave para la construcción de modelos y el proceso de aprendizaje. Los datos deben recopilarse, limpiarse, convertirse y luego enviarse al modelo para su aprendizaje.

El **ciclo de vida general de los datos** se muestra de la siguiente manera:



Uno de los requisitos críticos para el modelado es tener datos ‘buenos’ y ‘**equilibrados**’. Esto ayuda en modelos de mayor precisión y un mejor uso de los algoritmos disponibles. El tiempo de un científico de datos se dedica principalmente a la limpieza de los datos antes de construir el modelo.

Hemos visto el entrenamiento y las pruebas antes del despliegue del modelo. Para las pruebas, los resultados se capturan como métricas de evaluación, lo que nos ayuda a decidir si debemos usar un modelo en particular o cambiarlo.

A continuación, veremos las métricas de evaluación.

Métricas de evaluación

Evaluar un modelo implica verificar si el valor predicho es igual al valor real durante la fase de prueba. Hay varias métricas disponibles para verificar el modelo, y dependen del estado de la variable objetivo.

Para un problema de clasificación binaria, la variable objetivo predicha y la variable objetivo real pueden estar en cualquiera de los siguientes cuatro estados:

Predicha	Real
Predicha = TRUE	Real = TRUE
Predicha = TRUE	Real = FALSE
Predicha = FALSE	Real = TRUE
Predicha = FALSE	Real = FALSE

Cuando tenemos los valores predichos y reales como los mismos valores, se dice que somos precisos. Si todos los valores predichos y reales son iguales (ya sea VERDADERO o FALSO), el modelo es 100 por ciento exacto. Pero, este nunca es el caso.

Dado que las redes neuronales son modelos de aproximación, siempre hay un posible error. Los cuatro estados mencionados en la tabla anterior son posibles.

Definimos la siguiente terminología y métrica para un modelo:

- **Positivos verdaderos (PV)**: todos los casos en los que lo predicho y lo real son VERDADEROS (buena precisión).
- **Verdadero negativo (VN)**: todos los casos cuando se predice es FALSO y el real también es FALSO (buena precisión).
- **Falso positivo (FP)**: este es un caso cuando predcimos algo como positivo (VERDADERO), pero en realidad es negativo. Es como una falsa alarma o un error de FP. Un ejemplo es cuando se predice que un hombre está embarazado con un kit de prueba de embarazo. Todos los casos cuando se predicen son VERDADERO, mientras que el real es FALSO. Esto también se llama error de tipo 1.

- **Falso negativo (FN):** cuando predecimos algo como FALSO, pero en realidad es VERDADERO, entonces el caso se llama FN. Por ejemplo, cuando se predice que una mujer embarazada no está embarazada con un kit de prueba de embarazo, es un caso de FN. Todos los casos cuando se predicen son FALSO y los reales VERDADERO. Esto también se llama error de tipo 2.

Matriz de confusión

Cuando los valores de la clasificación se grafican en una matriz $n \times n$ (2×2 en caso de clasificación binaria), la matriz se llama matriz de confusión. Todas las métricas de evaluación pueden derivarse de la propia matriz de confusión:

	Valor predicho	Valor predicho
Valores reales	TRUE	FALSE
TRUE	PV	FN
FALSE	FP	VN

Ahora, veamos algunas métricas de evaluación en detalle.

Tasa positiva verdadera

La tasa positiva verdadera (TPV) o la sensibilidad o la tasa de recuerdo o acierto es una medida de cuántos positivos verdaderos se identificaron de todos los positivos identificados:

$$TPV = \frac{PV}{P} = \frac{PV}{PV + FN}$$

Idealmente, el modelo es mejor si tenemos esto más cerca de uno.

Tasa Negativa Verdadera

La Tasa Negativa Verdadera (TNV) o la especificidad es la proporción de negativos verdaderos y el número total de negativos que hemos predicho:

$$TNV = \frac{VN}{N} = \frac{VN}{VN + FP}$$

Si esta relación es más cercana a cero, el modelo es más preciso.

Exactitud (Precisión, accuracy)

La precisión es la medida de cuán bueno es nuestro modelo. Se espera que esté **más cerca de 1**, si nuestro modelo está funcionando bien.

La precisión es la proporción de predicciones correctas y todas las predicciones totales:

$$ACC = \frac{PV + VN}{P + N} = \frac{PV + VN}{PV + VN + FP + FN}$$

Precisión y recuerdo

La precisión y la recuperación son nuevamente relaciones entre los PV con (PV + FP) y los PV con (PV + FN) respectivamente. Estas proporciones determinan cuán relevantes se comparan nuestras predicciones con las reales.

La precisión se define como cuántos elementos seleccionados son relevantes. Es decir, **cuántos de los pronosticados se predicen realmente correctamente.**

La ecuación es:

$$Precision = \frac{PV}{PV + FP}$$

Si la precisión está más cerca de uno, somos más precisos en nuestras predicciones.

Recordemos, por otro lado, que indica cuántos elementos relevantes seleccionamos.

Matemáticamente, es:

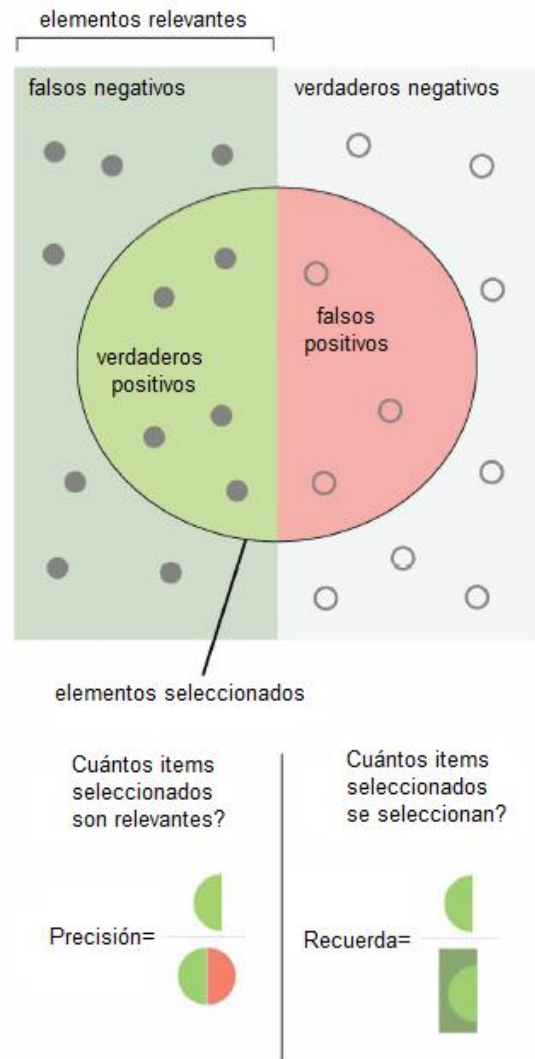
$$Recuerdo = \frac{PV}{PV + FN}$$

El siguiente diagrama muestra claramente la discusión que hemos hecho hasta ahora:

Puntaje F (F-score)

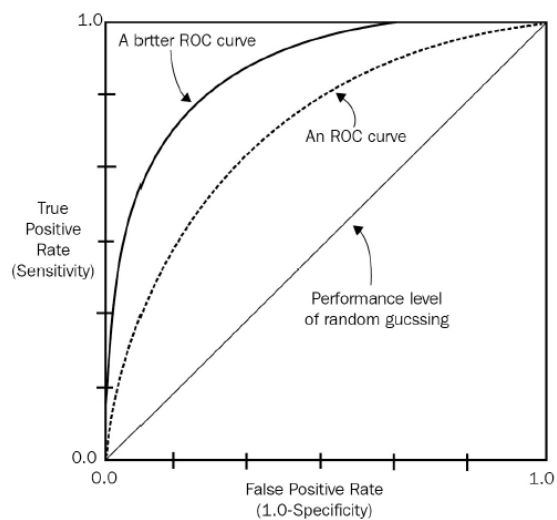
F-score, o F1-score, es otra medida de precisión. Técnicamente, **es la media armónica de precisión y recuerdo:**

$$F = 2 \cdot \frac{precision \cdot recuerdo}{precision + recuerdo}$$



Curva característica de funcionamiento del receptor

La curva de características operativas del receptor (Receiver Operating Characteristic, **ROC**) es una representación gráfica que **ilustra la capacidad predictiva de un sistema clasificador binario**. La curva ROC **se crea trazando un gráfico del TPV contra la tasa de falsos positivos (TFP) en varias configuraciones de umbral**. **Esto nos da sensibilidad versus (1 - especificidad)**. Una curva ROC generalmente se ve así:



Después de adquirir las habilidades necesarias, estamos listos para analizar en detalle los algoritmos utilizados para construir las redes neuronales.

Aprendizaje en redes neuronales

Como vimos en los primeros acetatos de esta parte del curso, **las redes neuronales son un algoritmo de aprendizaje automático que tiene la capacidad de aprender de los datos y darnos predicciones usando el modelo construido**. Es una aproximación de función universal, es decir, cualquier entrada, salida de datos puede aproximarse a una función matemática.

La propagación directa nos da una función matemática inicial para llegar a la(s) salida(s) en función de las entradas al elegir pesos aleatorios. La diferencia entre lo real y lo previsto se denomina **término de error**. El proceso de aprendizaje en una red neuronal de retroalimentación (feed-forward) en realidad ocurre durante la etapa de propagación hacia atrás (backpropagation). El modelo se ajusta con los pesos al reducir el término de error en cada iteración. El descenso de gradiente se utiliza en el proceso de retropropagación.

Revisaremos la propagación hacia atrás en detalle en esta parte del curso, ya que es un aspecto importante del aprendizaje automático para redes neuronales.

Regresemos a la propagación hacia atrás

Hemos cubierto la propagación hacia adelante en detalle en la primera parte del curso de RNA, y un poco sobre la propagación hacia atrás utilizando el descenso de gradiente. **La retropropagación** es uno de los conceptos importantes para comprender las redes neuronales y **se basa en el cálculo para actualizar los pesos y los sesgos en cada capa**.

La propagación hacia atrás de los errores es similar a aprender de los errores. Nos corregimos en nuestras equivocaciones (errores) en cada iteración, hasta llegar a un punto llamado **convergencia**. El objetivo de la retropropagación es corregir los pesos en cada capa y minimizar el error general en la capa de salida.

El aprendizaje de redes neuronales depende en gran medida de la propagación hacia atrás en las redes de retroalimentación. Los pasos habituales de propagación hacia adelante y corrección de errores se explican a continuación:

1. Iniciar la propagación hacia adelante de la red neuronal asignando pesos aleatorios y sesgos a cada una de las neuronas en la capa oculta.
2. Obtener la suma de $\text{suma}(\text{peso} * \text{entrada}) + \text{sesgo}$ en cada neurona.
3. Aplicar la función de activación (sigmoide) en cada neurona.
4. Tomar este resultado y pasarlo a la siguiente neurona de capa.

5. Si la capa es la capa de salida, aplicar los pesos y obtener la suma de suma(peso * entrada) + sesgo en cada neurona de la capa de salida.
6. Nuevamente, aplicar la función de activación en la neurona de la capa de salida.
7. Esto forma la salida de la red neuronal en la capa de salida para un paso directo.
8. Ahora, con los datos de entrenamiento, podemos identificar el término de error en cada neurona de salida, restando la salida real y el valor de salida de la función de activación.
9. Se llega al total de los errores utilizando la siguiente fórmula:

$$E = \frac{1}{2}(t - y)^2,$$

donde

E es el error al cuadrado,

t es el resultado objetivo para una muestra de entrenamiento, y

y es la salida real de la neurona de salida.

Se utiliza un factor de 1/2 para cancelar el exponente cuando la función de error E se diferencia posteriormente.

10. La técnica de descenso de gradiente requiere el cálculo de la derivada parcial del término del error (E) con respecto a los pesos de la red. El cálculo de la derivada parcial del error completo con respecto al peso w_{ij} se realiza utilizando la regla de la cadena de diferenciación:

$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial o_j} \frac{\partial o_j}{\partial net_j} \frac{\partial net_j}{\partial w_{ij}}$$

La derivada se define como la tasa de cambio de un valor, el descenso del gradiente usa la derivada (o pendiente) para minimizar el término de error y llegar a un conjunto correcto de pesos.

11. El primer factor es una derivada parcial del término del error con respecto a la salida en esa neurona particular j y o_j es igual a y :

$$\frac{\partial E}{\partial o_j} = \frac{\partial E}{\partial y} \frac{\partial}{\partial y} \frac{1}{2}(t - y)^2 = y - t$$

12. El segundo factor en la regla de la cadena es la derivada parcial de la salida de la neurona o_j con respecto a su entrada, y es la derivada parcial de la función de activación (la función sigmoide):

$$\frac{\partial o_j}{\partial net_j} = \frac{\partial}{\partial net_j} \varphi(net_j) = \varphi(net_j)(1 - \varphi(net_j))$$

Aquí net_j es la entrada a la neurona.

13. El tercer término en la regla de la cadena es simplemente o_i .

$$\frac{\partial net_j}{\partial w_{ij}} \frac{\partial}{\partial w_{ij}} \left(\sum_{k=1}^n w_{kj} o_k \right) = \frac{\partial}{\partial w_{ij}} w_{kj} o_k = o_i$$

14. Combinando los pasos 11, 12 y 13, tendremos:

$$\frac{\partial E}{\partial w_{ij}} = o_i \delta_j$$

con

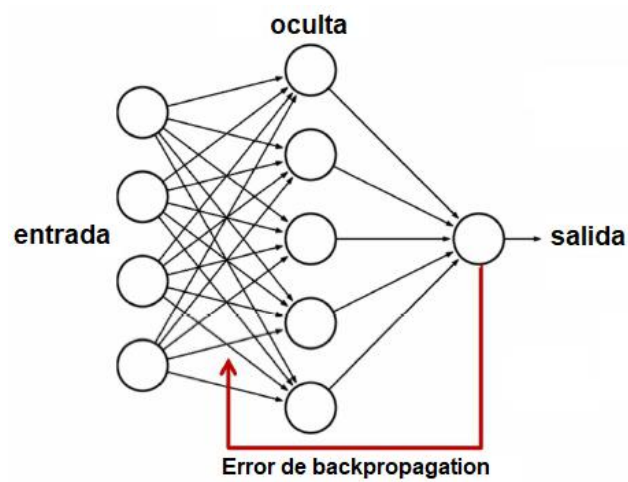
$$\delta_j = \frac{\partial E}{\partial o_j} \frac{\partial o_j}{\partial net_j} = \begin{cases} (o_j - t_j) o_j (1 - o_j) & \text{Si } j \text{ es una neurona de salida,} \\ \left(\sum_{l \in L} \delta_l w_{jl} \right) o_j (1 - o_j) & \text{Si } j \text{ es una neurona interna.} \end{cases}$$

15. El peso w_{ij} en cada neurona (cualquier capa) se actualiza con esta derivada parcial, combinada con la tasa de aprendizaje.

Estos pasos se repiten hasta que tengamos convergencia de un término de error muy bajo o un número específico de veces.

Todos los pasos se realizan internamente en los paquetes de R disponibles. Podemos suministrar la tasa de aprendizaje junto con varios otros parámetros.

La propagación hacia atrás se ilustra de la siguiente manera:



Como con todas las cosas en la vida, incluso un algoritmo tiene márgenes de mejora adicionales. En la siguiente sección, veremos cómo hacerlo.

Optimización del algoritmo de aprendizaje de redes neuronales

El procedimiento utilizado para llevar a cabo el proceso de aprendizaje en una red neuronal se llama algoritmo de entrenamiento. El algoritmo de aprendizaje es lo que el algoritmo de aprendizaje automático elige como modelo con la mejor optimización. El objetivo es minimizar la función de pérdida y proporcionar más precisión. Aquí ilustramos algunas de las técnicas de optimización, además del descenso de gradiente.

El método de optimización de enjambre de partículas (Particle Swarm Optimization, PSO) se inspira en observaciones del comportamiento social y colectivo sobre los movimientos de las bandadas de aves en busca de alimento o supervivencia. Es similar a una escuela de peces tratando de moverse juntos. Conocemos la posición y la velocidad de las partículas, y PSO apunta a buscar un conjunto de soluciones en un gran espacio controlado por ecuaciones matemáticas de posición y velocidad. Está inspirado en el comportamiento biológico del organismo para la inteligencia colectiva.

El recocido simulado (Simulated annealing) es un método que funciona en un enfoque probabilístico para aproximar el óptimo global para la función de costo. El método busca una solución en un gran espacio con simulación.

Los métodos evolutivos se derivan del proceso evolutivo en biología, y la evolución puede ser en términos de reproducción, mutación, selección y recombinación. Se utiliza una función de aptitud para determinar el rendimiento de un modelo y, en función de esta función, seleccionamos nuestro modelo final.

El método de Maximización de Expectativas (Expectation Maximization, EM) es un método de aprendizaje estadístico que utiliza un método iterativo para encontrar la probabilidad máxima o la estimación posterior máxima, minimizando así el error.

Aprendizaje supervisado en redes neuronales.

Como se mencionó anteriormente, el aprendizaje supervisado es un método de aprendizaje en el que hay una parte de los datos de entrenamiento que actúa como un maestro (o experto) para el algoritmo para determinar el modelo.

A continuación, se propone un ejemplo de un problema de modelado predictivo de regresión para comprender cómo resolverlo con redes neuronales.

Boston dataset

El conjunto de datos describe 13 propiedades numéricas de las casas en los suburbios de Boston, y se preocupa por modelar el precio de las casas en esos suburbios en miles de dólares. Como tal, **este es un problema de regresión de modelos predictivos**. Los atributos de entrada incluyen cosas como la tasa de criminalidad, la proporción de acres de negocios no minoristas, las concentraciones químicas y más. En la siguiente lista se muestran todas las variables seguidas de una breve descripción:

- Número de instancias: 506
- Número de atributos: 13 atributos continuos (incluido el atributo de clase MEDV) y un atributo de valor binario

Cada uno de los atributos se detalla a continuación:

1. tasa de criminalidad per cápita por ciudad (**crim**).
2. **zn** proporción de tierra residencial dividida en zonas para lotes de más de 25,000 pies cuadrados.
3. proporción industrial de acres de negocios no minoristas por ciudad (**indus**).
4. **chas** variable ficticia Charles River (= 1 si el tramo limita con el río; 0 en caso contrario).
5. concentración de óxidos nítricos, **nox** (partes por 10 millones).
6. **rm** número promedio de habitaciones por vivienda.
7. proporción de edad de las unidades ocupadas por el propietario construidas antes de 1940 (**age**).
8. distancias ponderadas a cinco centros de empleo de Boston (**dis**).
9. índice de accesibilidad a autopistas radiales (**rad**).
10. tasa impositiva del impuesto a la propiedad de valor total por \$10,000 (**tax**).
11. proporción de alumnos por profesor por ciudad (**ptratio**).
12. **black** $1000 (Bk - 0.63)^2$ donde **Bk** es la proporción de negros por ciudad.
13. estado inferior en porcentaje de la población (**lstat**).
14. valor medio promedio de viviendas ocupadas por sus propietarios en \$1000 (**medv**).

De estos, **medv es la variable de respuesta**, mientras que las otras trece variables son posibles predictores. El objetivo de este análisis es ajustar un modelo de regresión que mejor explique la variación en medv.

Existe una relación entre las primeras trece columnas y la variable de respuesta medv. Podemos predecir el valor medv en base a las trece columnas de entrada.

Nota: Este conjunto de datos ya se proporciona con las bibliotecas R (MASS), como veremos más adelante, por lo que no tenemos que preocuparnos por recuperar los datos.

Regresión con una red neuronal con el conjunto de datos de Boston

En esta sección, ejecutaremos una red neuronal de regresión para el conjunto de datos de Boston. El valor medv se predice para los datos de prueba. El tren para probar la división es 70:30.

La función neuralnet se usa para modelar los datos con una red neuronal:

```
> install.packages("MASS")
> library("neuralnet")
> library(MASS)
> set.seed(1)

> data = Boston

> max_data <- apply(data, 2, max)
> min_data <- apply(data, 2, min)
> data_scaled <- scale(data, center = min_data, scale = max_data - min_data)

> index = sample(1:nrow(data), round(0.70*nrow(data)))
> train_data <- as.data.frame(data_scaled[index,])
> test_data <- as.data.frame(data_scaled[-index,])

> n = names(data)
> f = as.formula(paste("medv ~", paste(n[!n %in% "medv"], collapse = " + ")))
> net_data = neuralnet(f, data=train_data, hidden=10, linear.output=T)
> plot(net_data)

> predict_net_test <- compute(net_data, test_data[, 1:13])
> predict_net_test_start <- predict_net_test$net.result * (max(data$medv) -
min(data$medv)) + min(data$medv)
> test_start <- as.data.frame((test_data$medv) * (max(data$medv) -
min(data$medv)) + min(data$medv))
> MSE.net_data <- sum((test_start - predict_net_test_start)^2) / nrow(test_start)

> Regression_Model <- lm(medv ~ ., data=data)
> summary(Regression_Model)
> test <- data[-index,]
> predict_lm <- predict(Regression_Model, test)
> MSE.lm <- sum((predict_lm - test$medv)^2) / nrow(test)

> MSE.net_data
> MSE.lm
```

A continuación, explicamos línea por línea el código propuesto para este ejemplo.

```
> library("neuralnet")  
> library(MASS)
```

Las primeras dos líneas del código son simples, ya que cargan las bibliotecas que usaremos para cálculos posteriores.

Específicamente, la biblioteca de red neuronal nos ayudará a construir y entrenar la red, mientras que la biblioteca MASS nos servirá para cargar el conjunto de datos de Boston que hemos introducido anteriormente en detalle.

Consejo: Recuerda, para instalar una biblioteca que no está presente en la distribución inicial de R, debes usar la función `install.package`. Esta es la función principal para instalar paquetes. Toma un vector de nombres y una biblioteca de destino, descarga los paquetes de los repositorios y los instala.

En nuestro caso, por ejemplo, para instalar el paquete `neuralnet`, debemos escribir:
`install.neuralnet` Aunque ya la habíamos instalado previamente!

Finalmente, se debe enfatizar que esta función debe usarse solo una vez y no cada vez que ejecutes el código.

En su lugar, carga la biblioteca a través del siguiente comando y debe repetirse cada vez que ejecutes el código:

```
library (neuralnet)
```

La función `set.seed` establece la semilla del generador de números aleatorios de R, que es útil para crear simulaciones u objetos aleatorios que se pueden reproducir:
`set.seed (1)`

Consejo: debes usar esta función cada vez que desees obtener un resultado aleatorio reproducible. En este caso, los números aleatorios son los mismos, y continuarían siendo los mismos sin importar qué tan lejos avancemos en la secuencia.

El siguiente comando carga el conjunto de datos de Boston, que, como anticipamos, está contenido en la biblioteca MASS y lo guarda en un marco dado:

```
data = Boston
```

Usa la función `str` para ver de forma compacta la estructura de un objeto R arbitrario. En nuestro caso, utilizando `str (data)`, obtendremos los siguientes resultados:

```
> str(data)
'data.frame': 506 obs. of 14 variables:
 $ crim : num 0.00632 0.02731 0.02729 0.03237 0.06905 ...
 $ zn : num 18 0 0 0 0 0 12.5 12.5 12.5 12.5 ...
 $ indus : num 2.31 7.07 7.07 2.18 2.18 2.18 7.87 7.87 7.87 7.87 ...
 $ chas : int 0 0 0 0 0 0 0 0 0 0 ...
 $ nox : num 0.538 0.469 0.469 0.458 0.458 0.458 0.524 0.524 0.524 0.524 ...
 $ rm : num 6.58 6.42 7.18 7 7.15 ...
 $ age : num 65.2 78.9 61.1 45.8 54.2 58.7 66.6 96.1 100 85.9 ...
 $ dis : num 4.09 4.97 4.97 6.06 6.06 ...
 $ rad : int 1 2 2 3 3 3 5 5 5 5 ...
 $ tax : num 296 242 242 222 222 222 311 311 311 311 ...
 $ ptratio: num 15.3 17.8 17.8 18.7 18.7 18.7 15.2 15.2 15.2 15.2 ...
 $ black : num 397 397 393 395 397 ...
 $ lstat : num 4.98 9.14 4.03 2.94 5.33 ...
 $ medv : num 24 21.6 34.7 33.4 36.2 28.7 22.9 27.1 16.5 18.9 ...
```

```
> max_data <- apply(data, 2, max)
> min_data <- apply(data, 2, min)
> data_scaled <- scale(data, center = min_data, scale = max_data - min_data)
```

Necesitamos este fragmento de código para **normalizar los datos**.

Consejo: Recuerda, **es una buena práctica normalizar los datos antes de entrenar una red neuronal**. **Con la normalización, se eliminan las unidades de datos, lo que le permite comparar fácilmente los datos de diferentes ubicaciones**.

Este es un procedimiento extremadamente importante en la construcción de una red neuronal, ya que evita resultados innecesarios o procesos de entrenamiento muy difíciles que resultan en problemas de convergencia de algoritmos. Puedes elegir **diferentes métodos para escalar los datos** (z-normalization, min-max scale, etc.). Para este ejemplo, utilizaremos el método minmax (generalmente llamado **escalamiento de características**) para obtener todos los datos escalados en el rango [0,1]. La fórmula para lograr esto es la siguiente:

$$x_{escalada} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Antes de aplicar el método elegido para la normalización, debes **calcular los valores mínimo y máximo de cada columna de la base de datos**. Para hacer esto, usamos la función `apply`. Esta función devuelve un vector o una matriz o una lista de valores obtenidos al aplicar una función a los márgenes de una matriz o matriz. Comprendamos el significado de los argumentos utilizados.

```
> max_data <- apply(data, 2, max)
```

El primer argumento de la función de aplicación especifica el conjunto de datos para aplicar la función, en nuestro caso, el conjunto de datos denominado `data`. El segundo argumento debe contener un vector que proporcione los subíndices sobre los cuales se aplicará la

función. En nuestro caso, **uno indica filas y 2 indica columnas**. El tercer argumento debe contener la función que se aplicará; en nuestro caso, la función max.

Para normalizar los datos, utilizamos la función de escala (**scale**), que **es una función genérica cuyo método predeterminado centra y/o escala las columnas de una matriz numérica**.

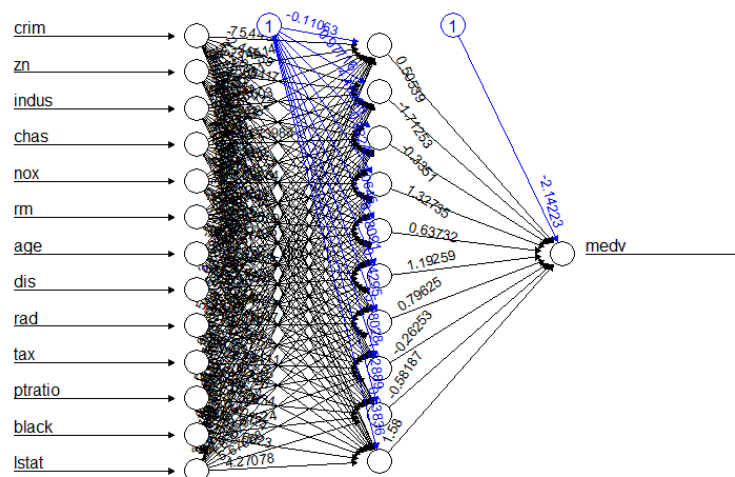
```
> index = sample(1:nrow(data),round(0.70*nrow(data)))
> train_data <- as.data.frame(data_scaled[index,])
> test_data <- as.data.frame(data_scaled[-index,])
```

En la primera línea del código que acabamos de sugerir, el conjunto de datos se divide en 70:30, con la intención de utilizar el 70 por ciento de los datos a nuestra disposición para entrenar a la red y el 30 por ciento restante para probar la red. En la segunda y tercera línea, los datos del marco (dataframe) de datos denominado data se subdividen en dos nuevos marcos de datos, llamados train_data y test_data.

```
> n = names(data)
> f = as.formula(paste("medv ~", paste(n[!n %in% "medv"], collapse = " + ")))
> net_data = neuralnet(f,data=train_data,hidden=10,linear.output=T)
> plot(net_data)
```

Hasta ahora, todo el código propuesto solo se ha utilizado para preparar los datos. Ahora es el momento de construir la red. Para hacer esto, primero recuperamos todos **los nombres de variables usando la función de nombres (names)**. Esta función obtendrá o establecerá el nombre de un objeto.

A continuación, creamos una fórmula (formula) que usaremos para construir la red, por lo que usamos la función de red neuronal (neuralnet) para construir y entrenar la red. En este caso, crearemos una red con solo una capa oculta con 10 nodos. Finalmente, trazamos la red neuronal, como se muestra en la siguiente figura:



Ahora que tenemos la red, ¿qué hacemos? Por supuesto, la usamos para hacer predicciones. Habíamos reservado un 30 por ciento de los datos disponibles para hacer esto:

```
> predict_net_test <- compute(net_data, test_data[,1:13])
```

En nuestro caso, aplicamos la función al conjunto de datos test_data, utilizando solo las primeras 13 columnas que representan las variables de entrada de la red:

```
> predict_net_test_start <- predict_net_test$net.result*(max(data$medv)-
min(data$medv))+min(data$medv)
> test_start <- as.data.frame((test_data$medv)*(max(data$medv)-
min(data$medv))+min(data$medv))
> MSE.net_data <- sum((test_start - predict_net_test_start)^2)/nrow(test_start)
```

Pero, ¿cómo podemos determinar si los pronósticos que la red puede realizar son precisos? Podemos usar el error cuadrático medio (Mean Squared Error, MSE) como una medida de cuán lejos están nuestras predicciones de los datos reales.

En este sentido, vale la pena recordar que antes de construir la red habíamos normalizado los datos. Ahora, para poder comparar, necesitamos dar un paso atrás y volver a la posición inicial. Una vez que se restauran los valores del conjunto de datos, podemos calcular el MSE a través de la siguiente ecuación:

$$MSE = \frac{\sum_{i=1}^n (y_{predicha} - y_{real})^2}{n}$$

Bueno, hemos calculado MSE ahora con ¿con qué lo comparamos? Para tener una idea de la precisión de la predicción de la red, podemos construir un modelo de regresión lineal:

```
> Regression_Model <- lm(medv~., data=data)
> summary(Regression_Model)
> test <- data[-index,]
> predict_lm <- predict(Regression_Model, test)
> MSE.lm <- sum((predict_lm - test$medv)^2)/nrow(test)
```

Construimos un modelo de regresión lineal usando la función lm. Esta función se utiliza para ajustar modelos lineales. Se puede utilizar para realizar regresión, análisis de varianza de un solo estrato y análisis de covarianza. Para producir resúmenes de resultados de los resultados del ajuste del modelo obtenido, hemos utilizado la función de resumen (summary), que devuelve los siguientes resultados (ver la siguiente figura):

Además, para el modelo de regresión, calculamos la MSE media. Finalmente, para evaluar el rendimiento de la red, se compara con un modelo de regresión lineal múltiple calculado con la misma base de datos de la siguiente manera:

```
> MSE.net_data
> MSE.lm
```

```
> summary(Regression_Model)

Call:
lm(formula = medv ~ ., data = data)

Residuals:
    Min       1Q   Median       3Q      Max
-15.595  -2.730  -0.518   1.777   26.199

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)  3.646e+01  5.103e+00   7.144 3.28e-12 ***
crim         -1.080e-01  3.286e-02  -3.287 0.001087 **
zn           4.642e-02  1.373e-02   3.382 0.000778 ***
indus        2.056e-02  6.150e-02   0.334 0.738288
chas         2.687e+00  8.616e-01   3.118 0.001925 **
nox          -1.777e+01  3.820e+00  -4.651 4.25e-06 ***
rm           3.810e+00  4.179e-01   9.116 < 2e-16 ***
age          6.922e-04  1.321e-02   0.052 0.958229
dis          -1.476e+00  1.995e-01  -7.398 6.01e-13 ***
rad          3.060e-01  6.635e-02   4.613 5.07e-06 ***
tax          -1.233e-02  3.760e-03  -3.280 0.001112 **
ptratio      -9.527e-01  1.308e-01  -7.283 1.31e-12 ***
black        9.312e-03  2.686e-03   3.467 0.000573 ***
lstat        -5.248e-01  5.072e-02  -10.347 < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 4.745 on 492 degrees of freedom
Multiple R-squared:  0.7406,    Adjusted R-squared:  0.7338
F-statistic: 108.1 on 13 and 492 DF,  p-value: < 2.2e-16
```

Los resultados son:

```
> MSE.net_data
[1] 27.63821
> MSE.lm
[1] 24.58499
```

A partir del análisis de los resultados, es posible notar que la red neuronal tiene un MSE más bajo (o más alto) que el modelo de regresión lineal. Lo que nos lleva a comparar el éxito del uso de una red neuronal. En este caso los resultados no reflejan justamente este hecho, pero es una forma importante de realizar este tipo de cálculos.